Lab-11 Retrieval-Augmented Generation (RAG)

Name: Satyatma Chincholi

USN:1RVU23CSE414

Date: 10-03-2026

1. Introduction to RAG

Retrieval-Augmented Generation, or **RAG**, is a technique used to give Large Language Models (LLMs) access to data they weren't originally trained on.

Think of a standard LLM like a brilliant student taking an exam from memory. They are smart, but if the exam asks about a news event from yesterday or a private company document, they will guess (hallucinate). **RAG** is like giving that student an "open-book" exam where they can look up specific facts in a library before answering.

2. Description

Standard models like GPT-4 or Gemini have two major limitations:

1. **Knowledge Cutoff:** They don't know what happened after their training ended.
2. **Private Data:** They don't have access to your personal files or your company's internal database.

RAG solves this by "retrieving" relevant documents from an external source and "augmenting" the prompt with that information so the model can generate a factual, grounded response.

3. Architecture

The RAG architecture is typically split into two main phases: **Indexing** (Preparing the data) and **Retrieval & Generation** (The live process).

A. The Indexing Pipeline (Data Prep)

1. **Load:** Import documents (PDFs, Text, HTML).
2. **Split:** Break large documents into smaller "chunks" (since LLMs have a limited context window).
3. **Embed:** Convert those text chunks into numerical vectors (list of numbers) using an **Embedding Model**.
4. **Store:** Save these vectors in a **Vector Database** (like ChromaDB, Pinecone, or FAISS).

B. The Retrieval & Generation Pipeline (The App)

1. **Query:** The user asks a question.
2. **Retrieve:** The system converts the user's question into a vector and finds the most similar "chunks" in the Vector Database.

3. **Augment:** The system takes those chunks and the original question and puts them into a prompt template.
4. **Generate:** The LLM reads the provided context and answers the question.

Types of RAG:

16 types of RAG

Type	Key Feature	Benefits	Applications/ Requirements	Examples of Tooling / Library
Standard RAG	• Basic retrieval + generation integration • RAG-Sequence and RAG-Token variants	• Improved accuracy • Reduced hallucinations	• General-purpose QA systems • initial RAG implementations	• Hugging Foce Tronsformers • Focebook's RAGimplementation • LangChain
Agentic RAG	• Autonomous agents • Tool use • Dynamic retrieval	• Handles complex tasks • Proactive AI	• Personal assistonts • Research alds • Customer service bots needing dynamic interaction	• LangChain Agents • Openal GPT-4 with Plugins • Microsoft Semantic Kernal
Graph RAG	• Knowledge grophs • Relational reasoning	• Rich information • Strong context handling	• Expert systems (medicine,law, enginearing) • Semantic search engines	• Noo4j Groph Database • Apache Jena • Stardog
Modular RAG	• Independent modules for retrieval, reasoning generation	• Flexibility • Scalability	• Large colaborative projects • Systems needing frequent updates	• Microservices Architecture • Docker & Kuberetes • Apache Kafkg
Memory-Augmented RAG	• External memory storoge & retrieval	• Continuity • Personalization	• Chatbots maintaining long-term contaxt • Personalized recommendations	• Redis for session storage • Amazon DynamoDe • Pinecone Vector Database
Multi-Modal RAG	• Cross-modal retrieval (text, image, audio)	• Richer responses • Accessibility	• Image captioning • Video summarization • Multimodal assistants	• OpenAI CUP • TensorFlow Hub Models • PyTorch Multi-Modal Libraries
Federated RAG	• Decentralized data sources • Privacy-preserving learning	• Data security • Compliance	• Healthcare systems handling sensitive data • Cross-organization collaboration	• TensorFlow Federated • PySyft by OpenMined • Fedgrated Learning Libraries
Streaming RAG	• Real-time data retrieval & generation	• Up-to-date information • Low latency	• Live reporting • Financial tickers • Social media monitoring	• Apache Kafko Streams • Amazon Kinesis • Spark Streaming
ODQA RAG	• Brood knowledge base • Dynamic retrieval	• Broad applicability • Dynamic responses	• Search ongines • Virtual assistonts handing diverse queries	• Elasticsoarch • Haystack by deepsat • Hugging Face Transformers
Contextual Retrieval RAG	• Context-oware retrieval using conversation history	• Personalization • Coherence	• Conversational AI • Customer support chatibots mointaining session context	• Dialogflow by Google • Rasa Open Source • Microsoft Bot Framework
Enhanced RAG	• integration of structured knowledge bases	• Factual accuracy • Domain expertise	• Educational tools • Professional domain apps (legal medical	• Knowledge Groph Embedding Libraries • OWL APIs • Apache Jena
Domain-Specific RAG	• Customization for specific industries	• Relevance • Compliance Trustworthiness	• Logal research assistonts • Medical diagnosis support • Financiol analysis tools	• LexPredict Contract Analytics • Watson Hoalth • Financial NLP Tools
Hybrid RAG	• Combines multiple retrieval approaches	• Improved recall • Enhanced relevance	• Complex QA systems • Search engines needing lexical+ semantic matching	• Elosticsearch with kNN plugin • FAISS by Facebook AI • Hybrid Retrieval Libraries
Self-RAG	• Self-reflection mechanisms Iterative refinement	• Enhanced accuracy • Improved coherence	• Content creotion tools • High-accuracy educational platforms	• OpenAI GPT models with fine-tuning • Human-in-the-Loop platforms
HyDE RAG	• Hypothetical document embeddings for guided retrieval	• Better recall • Higher answer quality	• Complex queries with implicit • Research assistance in niche fields	• Custom Transformer implementations • Hoystack Pipelines
Recursive/Multi-Step RAG	• Multiple rounds of retrieval and generation	• Enhanced reasoning • Deeper understanding	• Analytical & probler-solving tasks • Multi-turn dialogue systems	• LangChoin choins & agents • DeepMind AlphaCode framework